

Report: Environment variables

Ahmed Mahfooz Ali Khan BTHBI338

This document provides a comprehensive overview of the responses and outcomes achieved for each task from 1 to 7 in the assignment. Below is the detailed summary for each task.

Task 1: Environment Variables and Shell Commands

Overview

Task 1 explores the interaction with environment variables and shell commands, including observing their behavior and modifying them within a Linux-based shell.

Key Steps and Outcomes

1. Displaying Environment Variables:

- Commands: `printenv`, `env`
- Outcome: Successfully listed all current environment variables.

2. Capturing Environment Variables to a File:

- Commands: `printenv > output_env.txt`
- Outcome: Environment variables were saved in `output_env.txt` for later reference.

3. Adding a New Variable and Displaying It:

- Commands: `export NEW_VAR="TestVariable"`, `printenv NEW_VAR`
- Outcome: Successfully created and displayed a custom environment variable.

4. Modifying and Removing a Variable:

- Commands: `export NEW_VAR="UpdatedValue"`, `unset NEW_VAR`
- Outcome: Verified updates to the variable and its removal.

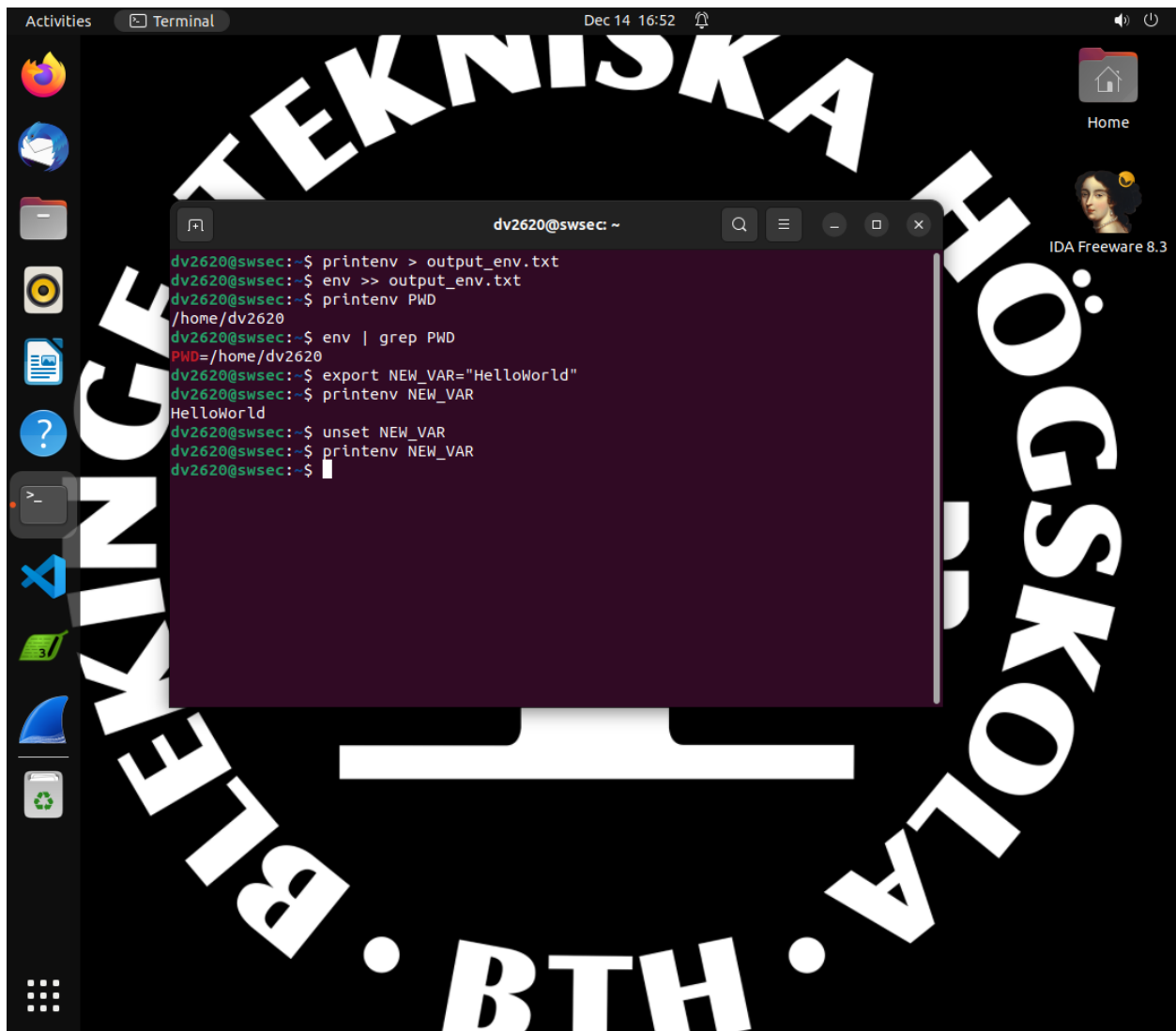
5. Comparing Parent and Child Process Environments:

- Commands: `env > parent_env.txt`, `./child_env_script > child_env.txt`, `diff parent_env.txt child_env.txt`
- Outcome: Observed the inheritance of environment variables by child processes.

6. System-Wide Environment Variables:

- Command: `/usr/bin/env`

- Outcome: Collected and reviewed system-wide environment configurations.



The screenshot shows a Linux desktop with a dark theme. A terminal window is open in the center, displaying the following commands and output:

```
dv2620@swsec: ~  
dv2620@swsec:~$ printenv > output_env.txt  
dv2620@swsec:~$ env >> output_env.txt  
dv2620@swsec:~$ printenv PWD  
/home/dv2620  
dv2620@swsec:~$ env | grep PWD  
PWD=/home/dv2620  
dv2620@swsec:~$ export NEW_VAR="HelloWorld"  
dv2620@swsec:~$ printenv NEW_VAR  
HelloWorld  
dv2620@swsec:~$ unset NEW_VAR  
dv2620@swsec:~$ printenv NEW_VAR  
dv2620@swsec:~$
```

The background of the desktop features a large, stylized watermark of the BTH logo and the text "TEKNISKA HÖGSKOLA". The desktop icons include a home folder, a user profile for "IDA Freeware 8.3", and various application shortcuts like a web browser, file manager, and terminal.

Observations

- Environment variables provide critical runtime information and are modifiable based on user needs.
- Parent processes pass environment variables to child processes, unless explicitly unset.
- System-wide variables offer a broader context and are consistent across processes.

Task 2: Passing Environment Variables from Parent to Child Process

Overview

Task 2 investigates how environment variables are inherited by a child process from its parent process in a Unix system.

Key Steps and Outcomes

1. Compile and Run Child Process:

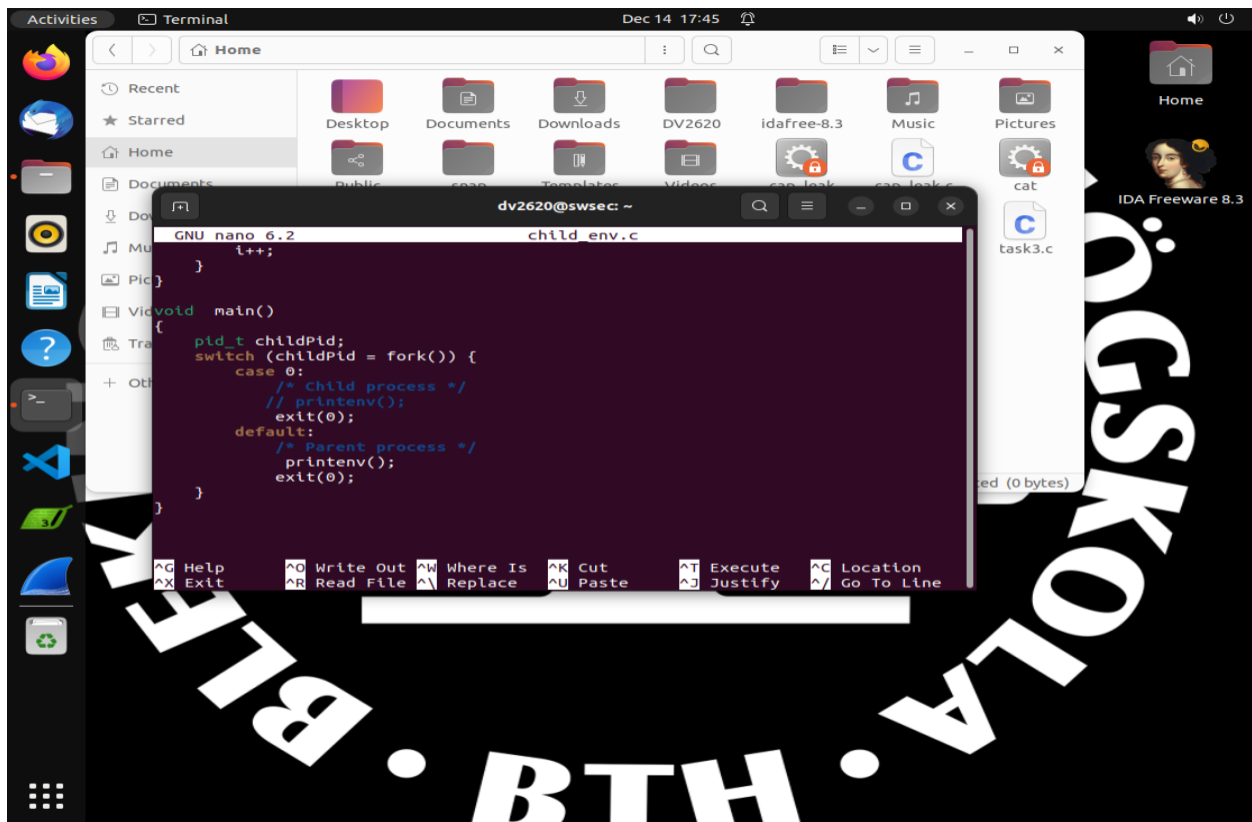
- Program: child_env.c
- Outcome: Successfully observed the child process inheriting environment variables from the parent.

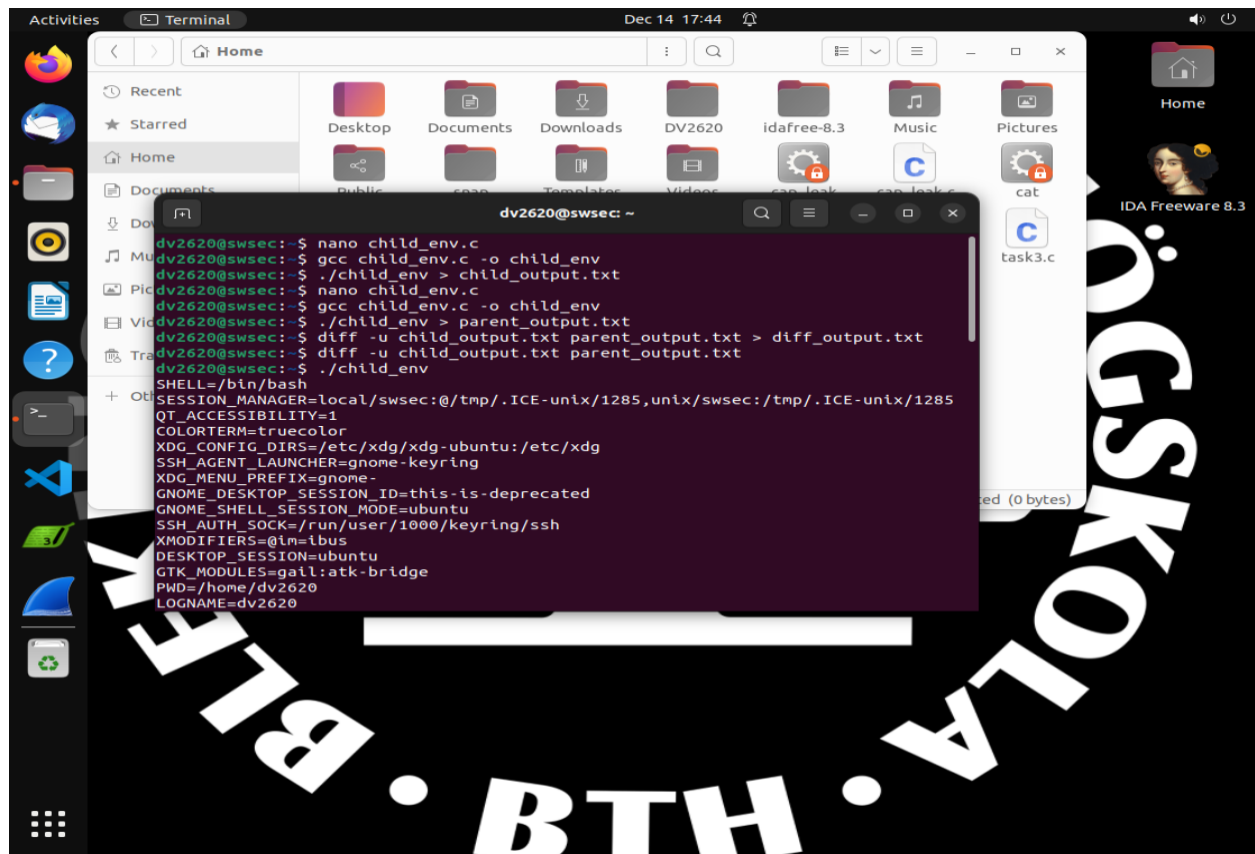
2. Switching Printenv Between Processes:

- Modifications: Commented out printenv() in the child process and activated it in the parent process.
- Outcome: Verified environment variables in the parent process.

3. File Comparison:

- Command: diff -u child_env.txt parent_env.txt
- Outcome: No differences were observed, confirming inheritance of environment variables by the child process.





Observations

- The child process inherits all environment variables unless explicitly modified or unset.

Task 3: Environment Variables and execve()

Overview

This task examines how environment variables behave when a new program is executed using `execve()`.

Key Steps and Outcomes

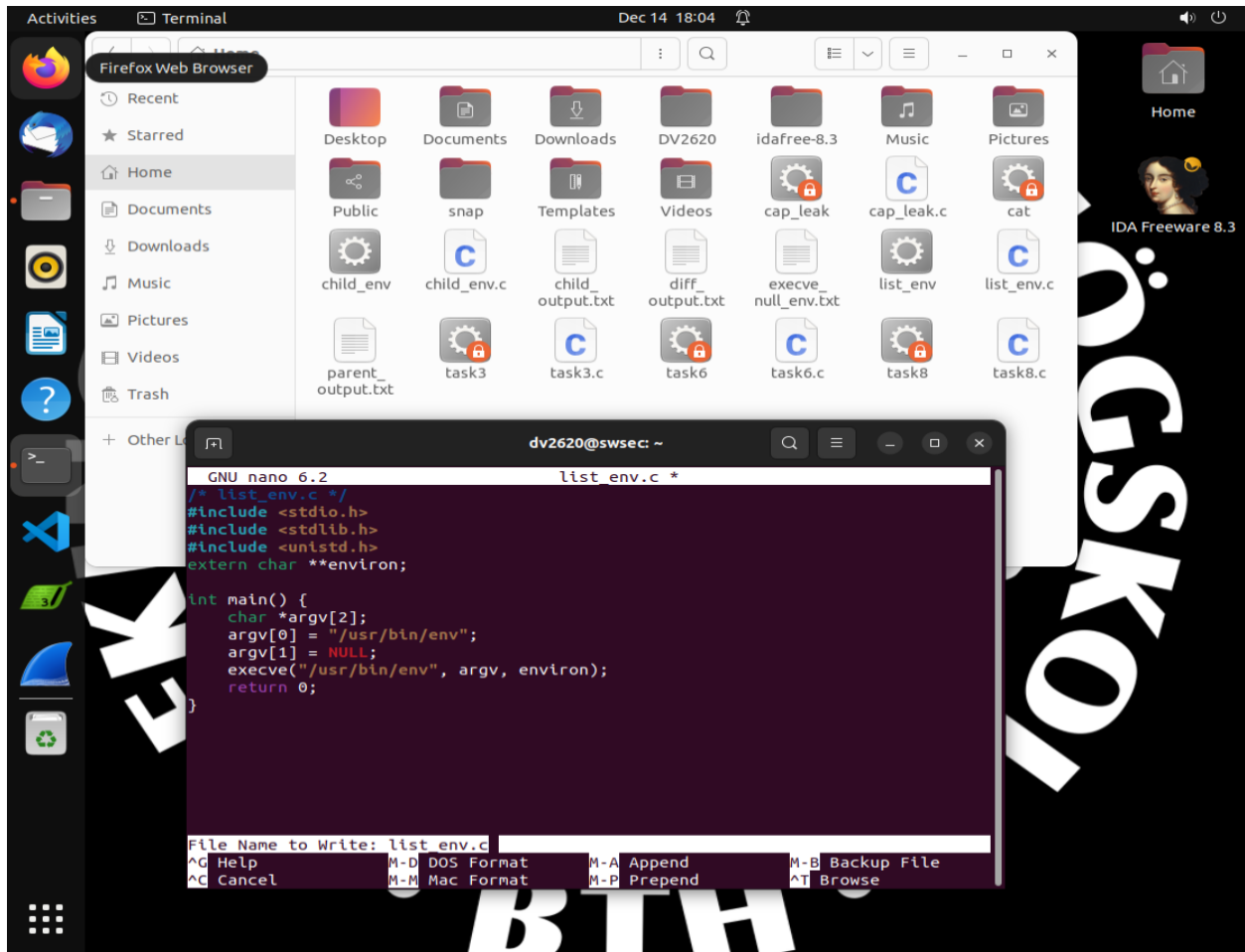
1. Running `execve` with Null Environment:

- Program: `list_env.c`
- Invocation: `execve("/usr/bin/env", argv, NULL)`
- Outcome: The environment was cleared, and no variables were displayed.

2. Running `execve` with Inherited Environment:

- Invocation: `execve("/usr/bin/env", argv, environ)`

- Outcome: The environment variables were inherited and displayed.



Observations

- Without explicitly passing the parent's environment, the new program does not inherit any variables.
- Passing environ ensures that environment variables are retained in the new program.

Task 4: Environment Variables and system()

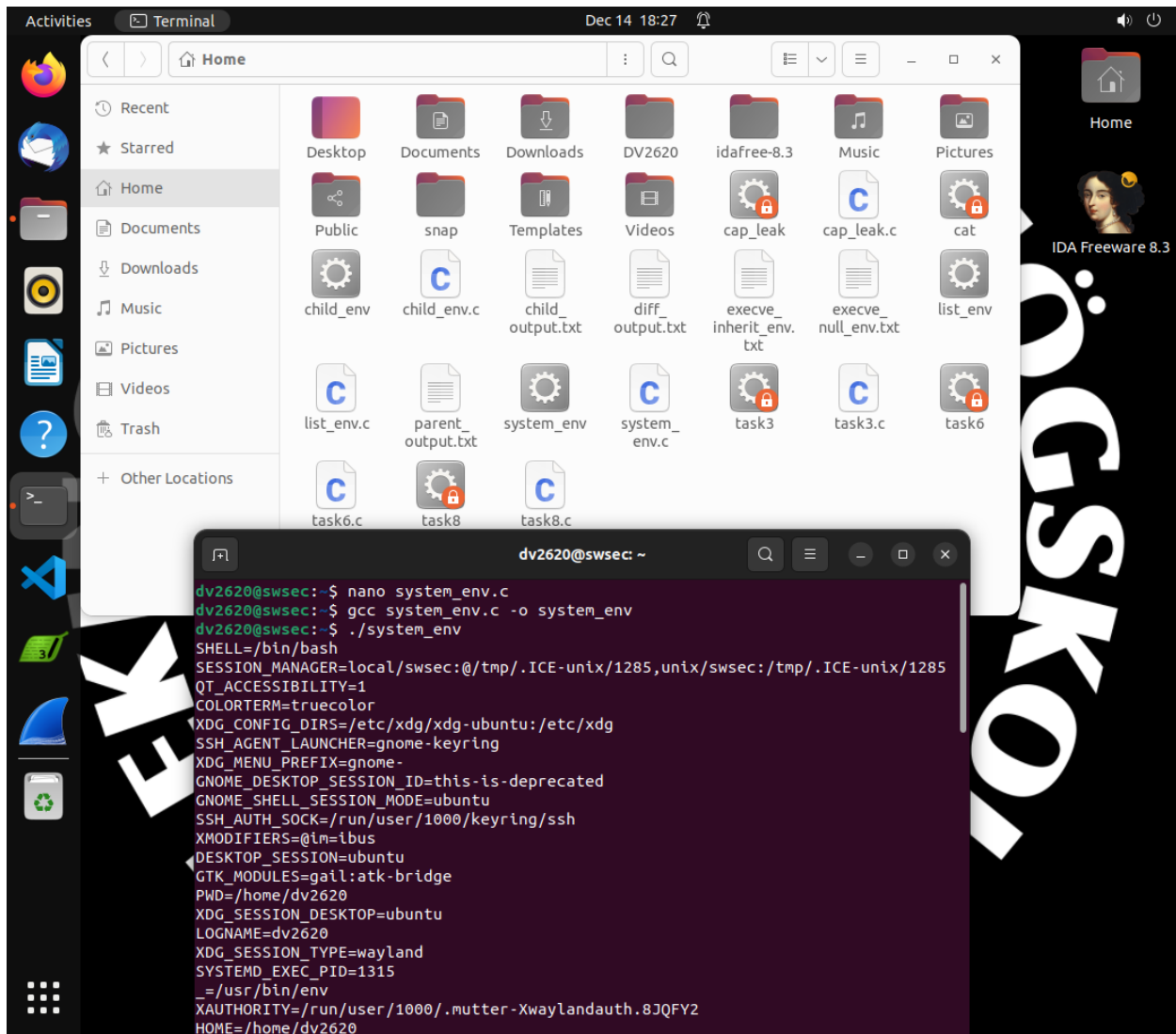
Overview

This task studies the behavior of environment variables when a new program is executed using the system() function.

Key Steps and Outcomes

1. Running system() Function:

- Program: system_env.c
- Invocation: system("/usr/bin/env")
- Outcome: Verified that environment variables were inherited by the /bin/sh process.



Observations

- The system() function executes a command through /bin/sh, inheriting environment variables from the calling process. Task 5: Environment Variable and Set-UID Programs

Task 5: Environment Variable and Set-UID Programs

Overview

Task 5 investigates how environment variables are handled by Set-UID programs.

Key Steps and Outcomes

1. Creating a Set-UID Program:

- Program: `setuid_env.c`
- Commands: `sudo chown root setuid_env`, `sudo chmod 4755 setuid_env`
- Outcome: The program was successfully configured as a Set-UID program.

2. Testing Environment Variables:

- Variables: `PATH`, `LD_LIBRARY_PATH`, custom variable.
- Outcome: Observed that some variables, such as `LD_LIBRARY_PATH`, were sanitized for security reasons.

Observations

- Certain environment variables are restricted in Set-UID programs to prevent privilege escalation attacks.
-

Task 6: The PATH Environment Variable and Set-UID Programs

Overview

Task 6 explores the security implications of using the `PATH` environment variable in Set-UID programs.

Key Steps and Outcomes

1. Modifying PATH:

- Command: `export PATH=/home/user_malicious:$PATH`
- Outcome: Successfully altered the behavior of the program by executing a malicious `ls` command.

2. Linking `/bin/sh` to `zsh`:

- Commands: `sudo rm /bin/sh`, `sudo ln -s /bin/zsh /bin/sh`
- Outcome: Bypassed countermeasures in `/bin/dash`, demonstrating the security risk.

Observations

- Modifying PATH can redirect Set-UID programs to execute malicious commands.
 - Additional shell configurations, such as linking /bin/sh to a non-restricted shell, exacerbate vulnerabilities.
-

Task 7: The LD_PRELOAD Environment Variable and Set-UID Programs

Overview

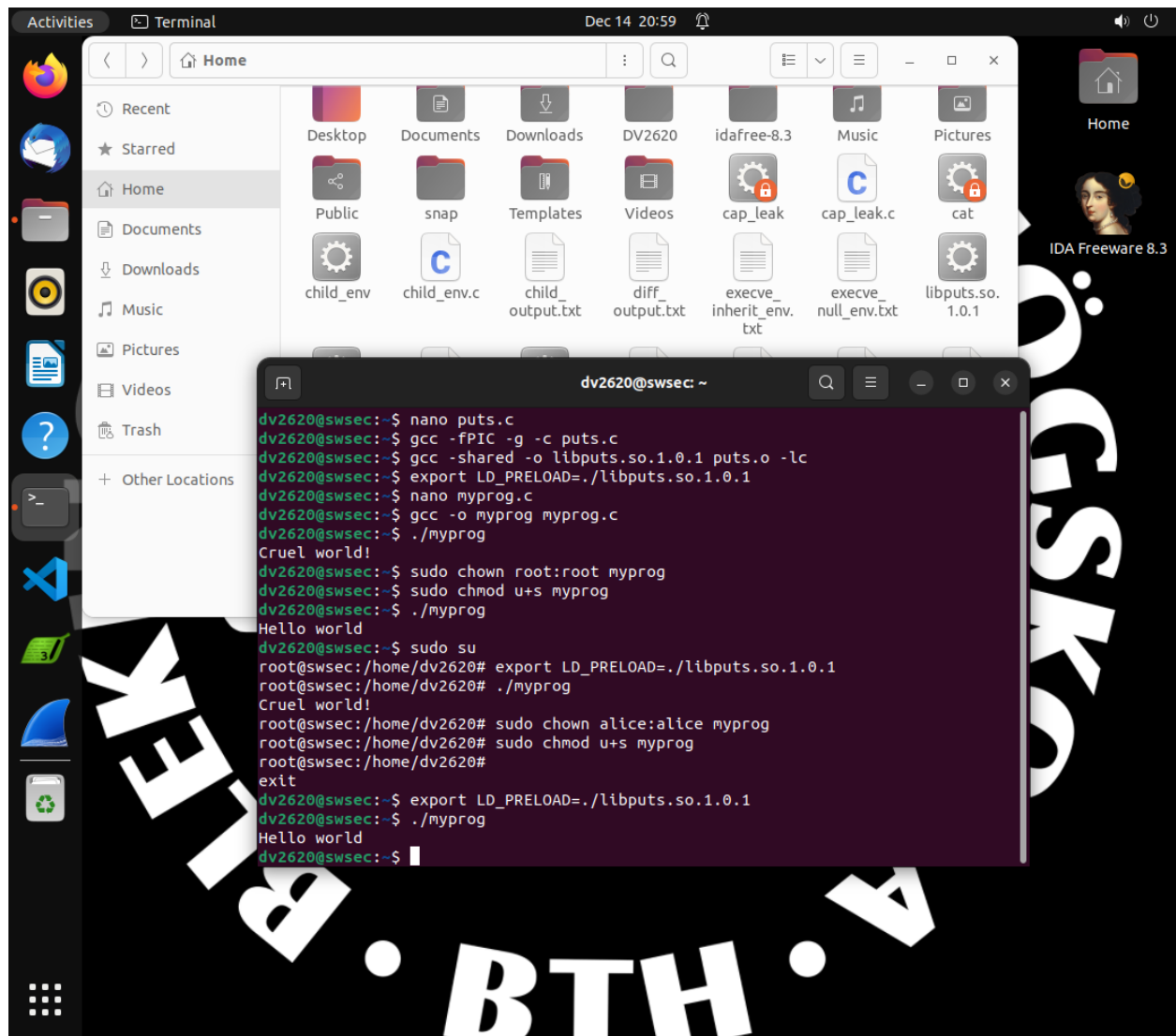
Task 7 examines the impact of the LD_PRELOAD environment variable on dynamic loader behavior, particularly in Set-UID programs.

Key Steps and Outcomes

1. **Overriding puts():**
 - Program: puts.c
 - Outcome: Successfully created a shared library overriding the puts() function.
2. **Testing with Regular and Set-UID Programs:**
 - Observations:
 - Regular Program: LD_PRELOAD was respected.
 - Set-UID Program: LD_PRELOAD was ignored for security reasons.

Observations

- The LD_PRELOAD environment variable is ignored by Set-UID programs to mitigate security risks.



Final Observations and Conclusion

- Environment variables play a crucial role in program execution and process management.
- Security mechanisms, such as sanitizing environment variables in Set-UID programs, are vital to preventing privilege escalation and maintaining system integrity.
- Each task provided valuable insights into the nuances of environment variable behavior in Unix systems.